



Ex04 – Symmetric Cryptography (Resumo Explicativo e Simplificado)

1. O que é Criptografia Simétrica

A **criptografia simétrica** é um método onde a **mesma chave** é usada para **encriptar** e **desencriptar** dados. É rápida e eficiente, sendo usada em grande parte das comunicações seguras atuais (como VPNs, discos encriptados, etc.).

O algoritmo mais usado é o **AES (Advanced Encryption Standard)**. O AES-128 utiliza blocos de 16 bytes e uma chave de 16 bytes (128 bits).

2. Modos de Operação (ECB e CBC)

Os modos de operação definem como os blocos de dados são processados durante a encriptação. O **AES**, por si só, apenas cifra um bloco de 16 bytes — os modos determinam como lidar com dados maiores.

- **ECB (Electronic Codebook)**: Cada bloco é cifrado separadamente. **Blocos iguais produzem cifras iguais** — por isso, é **inseguro** para dados com padrões (ex: imagens).
- **CBC (Cipher Block Chaining)**: Cada bloco é XOR com o bloco cifrado anterior antes de ser cifrado. Usa um **vetor de inicialização (IV)** no primeiro bloco. **É muito mais seguro.** 🍎

■ Em resumo: o **CBC elimina padrões visuais no texto cifrado e torna o resultado mais aleatório.**

3. Padding (Preenchimento)

Como o AES trabalha com blocos fixos de 16 bytes, é comum que o último bloco do texto não tenha o tamanho completo. Para resolver isso, **adiciona-se bytes extras até completar o bloco** — esse processo chama-se **padding**.

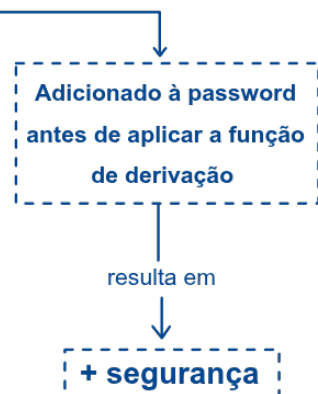
- **Padrão usado**: PKCS#7
- **Exemplo**: se faltarem 4 bytes, são adicionados quatro valores '0x04'.
- Durante a desencriptação, o padding é removido verificando o valor do último byte.
- Se o tamanho do texto for múltiplo de 16, adiciona-se um novo bloco completo de padding.

Porque cada bloco utiliza 16

4. Geração de Chaves e IV (PBKDF2)

As chaves AES devem ser aleatórias e seguras. Para gerar chaves a partir de uma senha humana, usa-se o **PBKDF2-HMAC-SHA256** — um algoritmo que deriva uma chave forte aplicando muitas iterações de hash sobre a senha e um **valor aleatório** chamado **salt**.

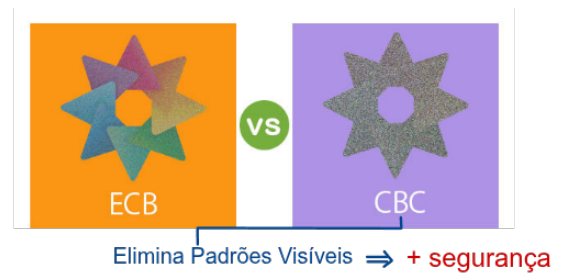
- **PBKDF2** = Password-Based Key Derivation Function 2
- **Digest**: SHA-256
- **Saída**: 256 bits (usados como chave e IV de 128 bits cada).
- **Comando em C**: pbkdf2-kickoff.c



5. Encriptação e Desencriptação de Ficheiros

Para encriptar ficheiros, o utilizador fornece:

- Uma **password** (usada para gerar a chave e IV).
- O **modo de operação** (ECB ou CBC).
- O **nome do ficheiro de entrada** e o **ficheiro de saída**.



A aplicação ¹ lê o ficheiro em blocos, ² aplica padding se necessário e ³ escreve o resultado cifrado. Durante a descriptação, o processo é invertido e o padding removido.

■ Questões típicas:

- O padding é visível após a descriptação? → Não, é removido automaticamente.
- O que acontece se o ficheiro cifrado estiver truncado? → A descriptação falha (dados inválidos).

↓
Incompleto após a encriptação

6. Padrões em Imagens (ECB vs CBC)

Para visualizar a diferença entre ECB e CBC, o exercício pede para cifrar uma imagem BMP. Depois, copia-se o cabeçalho (54 bytes) do ficheiro original para o ficheiro cifrado, para que a imagem possa ser aberta.

No modo **ECB**, padrões visuais da imagem original continuam visíveis no ficheiro cifrado. No modo **CBC**, a imagem cifrada parece totalmente aleatória — o que mostra a vantagem do encadeamento.

7. Perguntas possíveis (e respostas curtas)

- ■ O que é criptografia simétrica?
→ ■ Técnica que usa a mesma chave para cifrar e decifrar dados.
- ■ O que é o AES?
→ ■ Algoritmo de encriptação em blocos usado em padrões de segurança modernos.
- ■ Qual a diferença entre ECB e CBC?
→ ■ ECB cifra blocos isoladamente; CBC encadeia blocos (mais seguro).
- ■ O que é o IV? — Initialization Vector
→ ■ Vetor de inicialização usado no primeiro bloco do CBC para garantir aleatoriedade.
- ■ O que é PKCS#7?
→ ■ Método de padding que preenche o último bloco com bytes indicando quantos foram adicionados.
- ■ Para que serve o PBKDF2?
→ ■ Gera uma chave forte a partir de uma password e um salt.
- ■ O que acontece se o mesmo IV for usado duas vezes?
→ ■ A segurança é comprometida, pois padrões podem ser revelados.
- ■ Por que o ECB é inseguro?
→ ■ Porque blocos iguais de texto produzem blocos iguais de cifra,

o que torna o processo inverso muito mais fácil pois já conhecemos a "chave".

CBC

$$C_0 = \text{AES-eme}(P_0 \oplus IV)$$

1º Bloco (pointing to P₀) XOR (pointing to ⊕) Vetor Inicial (pointing to IV)

$$C_i = \text{AES-eme}(P_i \oplus C_{i-1})$$

iº Bloco (pointing to P_i) Bloco Anterior (pointing to C_{i-1})

Preparação Prática – Symmetric Cryptography (Guia 4)

Versão prática com perguntas abertas e de preencher código. Código em C quando o guião dá C; Python quando o guião sugere Python (cryptography). Respostas no fim.

Parte 1 – Exercícios (sem respostas)

1) AES-128-ECB (C, OpenSSL EVP) – preencher o ciclo

```
/* ecb_demo.c
 * Compilar: gcc -O2 ecb_demo.c -o ecb_demo -lcrypto
 */
#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

int main(void){
    unsigned char key[16] = {0}; /* PREENCHA com 16 bytes de teste */
    unsigned char in[16] = {0}; /* PREENCHA com 16 bytes de plaintext */
    unsigned char out[32]; int outlen=0, tmplen=0;

    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    EVP_EncryptInit_ex(ctx, EVP_aes_128_ecb(), NULL, key, NULL);
    EVP_CIPHER_CTX_set_padding(ctx, 0); /* ECB puro: sem padding */

    /* PREENCHA: chamar EVP_EncryptUpdate para cifrar 16 bytes */
    /* PREENCHA: finalizar com EVP_EncryptFinal_ex */
    /* PREENCHA: imprimir 'out' em hex (outlen bytes) */

    EVP_CIPHER_CTX_free(ctx);
    return 0;
}
```

Q1.1 Preencha as chamadas ao EVP_EncryptUpdate/Final e a impressão em hex.

Q1.2 Qual a consequência de ECB cifrar blocos iguais com a mesma saída?

2) AES-128-CBC (C, OpenSSL EVP) – com IV

```
/* cbc_demo.c
 * Compilar: gcc -O2 cbc_demo.c -o cbc_demo -lcrypto
 */
#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

int main(void){
    unsigned char key[16] = {0}; /* PREENCHA */
    unsigned char iv[16] = {0}; /* PREENCHA (não reutilizar IV!) */
    unsigned char in[16] = {0}; /* PREENCHA */
    unsigned char out[64]; int outlen=0, tmplen=0;

    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, key, iv);
    EVP_CIPHER_CTX_set_padding(ctx, 0); /* sem padding neste exemplo */
    EVP_EncryptUpdate(ctx, out, &outlen, in, 16);
}
```

```

EVP_EncryptFinal_ex(ctx, out+outlen, &tmlen); outlen += tmlen;

/* PREENCHA: imprimir out[0..outlen) em hex */
EVP_CIPHER_CTX_free(ctx);
return 0;
}

```

→ Porque ele usa blocos anteriores para cifrar e o ECB, para blocos iguais usa a mesma chave

Q2.1 Porque é que CBC evita padrões visíveis em imagens e ECB não?

3) PKCS#7 Padding (C) – implementar add/remove

```

/* pkcs7.c – implementar padding de 16 bytes */
#include <stdio.h>
#include <string.h>
#include <stdint.h>

size_t pkcs7_pad(unsigned char *buf, size_t len, size_t cap, size_t block){
    size_t rem = block - (len % block);
    if(rem==0) rem = block;
    if(len + rem > cap) return (size_t)-1;
    for(size_t i=0; i<rem; i++) buf[len+i] = (unsigned char)rem;
    return len + rem;
}

ssize_t pkcs7_unpad(unsigned char *buf, size_t len, size_t block){
    if(len==0 || len%block) return -1;
    unsigned char p = buf[len-1];
    if(p==0 || p>block) return -1;
    for(size_t i=0; i<p; i++) if(buf[len-1-i]!=p) return -1;
    return (ssize_t)(len - p);
}

```

→ Adiciona um bloco completo (16 bytes), para evitar ambigüidades

Q3.1 Explique o caso especial quando o input já é múltiplo do bloco.

Q3.2 O que deve fazer a função de unpad se o padding for inválido?

→ Deve abortar a criptografia

4) ECB/CBC com PKCS#7 (C, EVP) – completar

```

/* modes_pad.c */
#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

/* PREENCHA: usar pkcs7_pad antes de cifrar; pkcs7_unpad depois de decifrar */

int main(void){
    /* PREENCHA: ler dados para 'in', fazer pad, cifrar em ECB e CBC, e depois decifrar e remover padding */
    return 0;
}

```

Q4.1 Em que momento aplicas padding no fluxo de cifrar/decifrar?

É na parte da cifra, o unpadding é na decifra

5) PBKDF2-HMAC-SHA256 (C) – gerar 256 bits e dividir em key/IV

```

/* pbkdf2_kdf.c
 * Compilar: gcc -O2 pbkdf2_kdf.c -o pbkdf2_kdf -lcrypto
 */
#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

```

```

int main(void){
    const char *password = "secret";      /* PREENCHA: ler do utilizador */
    unsigned char salt[16] = {1,2,3,4,5}; /* PREENCHA: aleatório */
    int iters = 100000;
    unsigned char out[32];

    if(!PKCS5_PBKDF2_HMAC(password, strlen(password), salt, sizeof(salt), iters, EVP_sha256(), sizeof(out)))
        fprintf(stderr, "PBKDF2 fail\n"); return 1;
    }
    unsigned char key[16], iv[16];
    memcpy(key, out, 16);
    memcpy(iv, out+16, 16);

    /* PREENCHA: imprimir key e iv em hex */
    return 0;
}

```

Q5.1 Para que servem salt e iterations? E por que 256 bits para obter key+IV de 128+128?

Valor random, para garantir que o IV é único

6) File encryption/decryption (C, CBC+PKCS#7) – esqueleto

```

/* file_aes.c - cifrar/decifrar ficheiros com AES-128-CBC+PKCS#7
 * Compilar: gcc -O2 file_aes.c -o file_aes -lcrypto
 * Uso: ./file_aes enc|dec <password> <in> <out>
 */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <openssl/evp.h>
#include <openssl/rand.h>

int main(int argc, char **argv){
    if(argc!=5){ fprintf(stderr, "Uso: %s enc|dec <password> <in> <out>\n", argv[0]); return 1; }
    int enc = (strcmp(argv[1], "enc")==0);
    const char *password = argv[2];
    const char *fin = argv[3], *fout = argv[4];

    unsigned char salt[16]; RAND_bytes(salt, sizeof(salt));
    unsigned char dk[32];
    PKCS5_PBKDF2_HMAC(password, strlen(password), salt, sizeof(salt), 150000, EVP_sha256(), sizeof(dk));
    unsigned char key[16], iv[16]; memcpy(key, dk, 16); memcpy(iv, dk+16, 16);

    /* PREENCHA: abrir fin/fout; no enc gravar salt no início do out */
    /* PREENCHA: usar EVP_aes_128_cbc(), padding automático ON */
    /* PREENCHA: ciclo de EncryptUpdate/DecryptUpdate sobre blocos do ficheiro */
    /* PREENCHA: Final_ex e fechar ficheiros */

    return 0;
}

```

Q6.1 Como guardarias o salt no ficheiro de saída? E como o lês no decifrar?

Q6.2 O que acontece se o criptograma for truncado? E se faltar bytes no início?

7) (Python quando permitido) AES-128-CBC com padding e PBKDF2

```

# pip install cryptography
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes, padding

```

```

from cryptography.hazmat.backends import default_backend
import os

password = b"secret"
salt = os.urandom(16)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt, iterations=150_000)
dk = kdf.derive(password)
key, iv = dk[:16], dk[16:]

cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
padder = padding.PKCS7(128).padder()
padded = padder.update(b"plaintext") + padder.finalize()
enc = cipher.encryptor().update(padded) + cipher.encryptor().finalize()

```

Q7.1 Onde é aplicado o PKCS#7 nesta API? Como obténs key/IV a partir do dk?

8) Padrões (BMP) – ECB vs CBC

Depois de cifrar Tux.bmp com ECB para Tux_ecb.bmp, copiar cabeçalho BMP (54 bytes)
 dd if=Tux.bmp of=Tux_ecb.bmp ibs=1 count=54 conv=notrunc

Abrir Tux.bmp e Tux_ecb.bmp num visualizador e comparar
 # Repetir com CBC: Tux_cbc.bmp (mesmo procedimento)

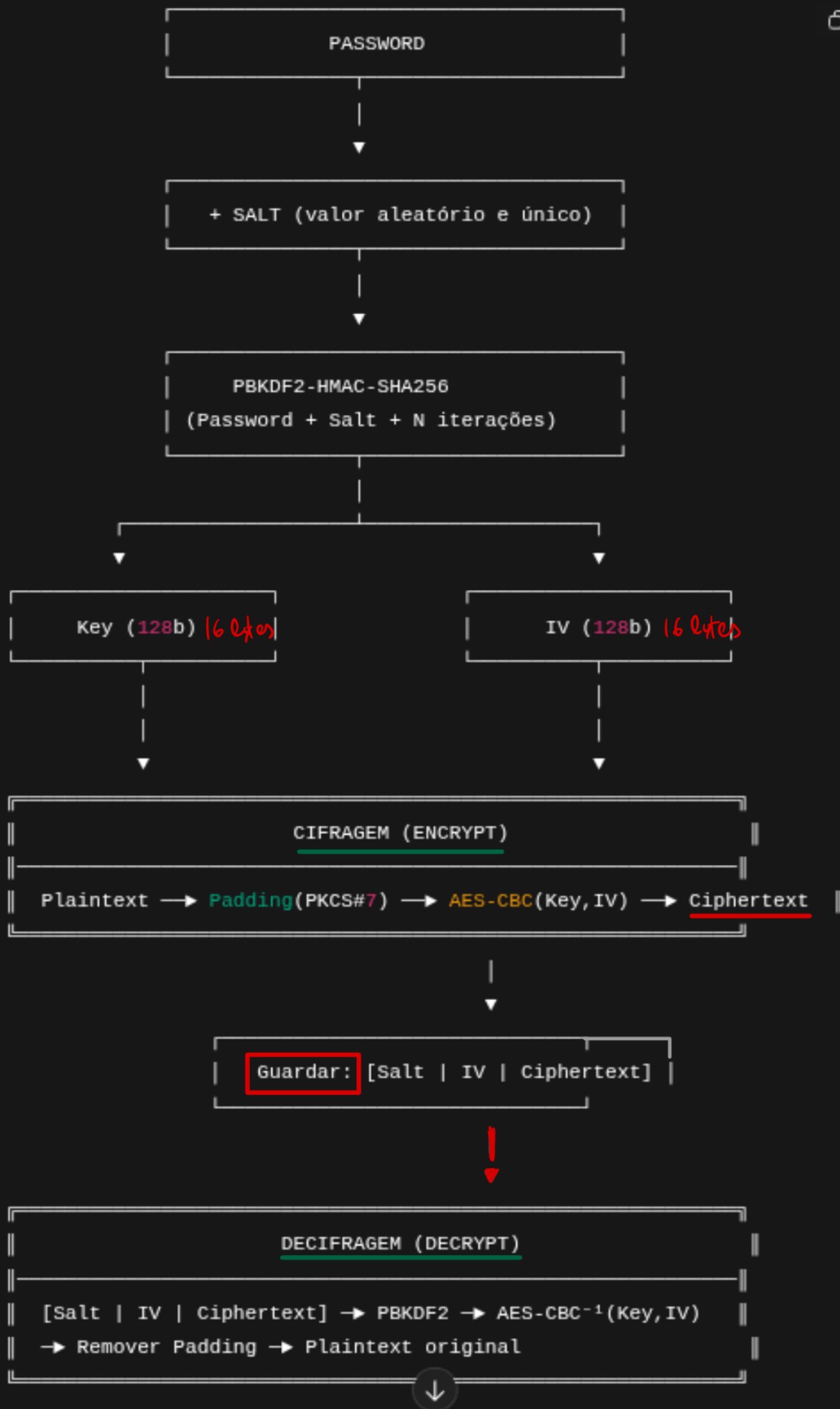
Q8.1 O que se observa na imagem cifrada em ECB? E em CBC?

Q8.2 Por que a reutilização de IV em CBC é perigosa?

R: Pode revelar padrões, que ~~podem revelar~~ ^{podem revelar} padrões e quebrar a segurança da cifra ma segurança da cifra

💡 Portanto:

Elemento	Papel	Vem de onde
Password	Entrada escolhida pelo utilizador	Inserida manualmente
Salt	Valor aleatório que entra no PBKDF2	Gerado aleatoriamente
PBKDF2	Função de derivação	Mistura password + salt
Key	• Originada pelo PBKDF2	Saída (1ª metade)
IV	• Originado pelo PBKDF2	Saída (2ª metade)



Parte 2 – Respostas

Q1.1

```
EVP_EncryptUpdate(ctx,out,&outlen,in,16);  
EVP_EncryptFinal_ex(ctx,out+outlen,&tmplen);  
outlen+=tmplen;  
for(int i=0;i<outlen;i++) printf("%02x", out[i]);
```

Q1.2

ECB produz ciphertexts idênticos para blocos de plaintext idênticos → padrões visíveis.

Q2.1

CBC encadeia blocos (XOR com IV/CT-1), quebrando padrões; ECB não encadeia.

Q3.1

Quando input é múltiplo do bloco, PKCS#7 adiciona um bloco extra (valor=16).

Q3.2

Falhar e não remover padding (retornar erro).

Q4.1

Aplicar padding antes de cifrar; remover padding depois de decifrar e validar.

Q5.1

Salt aleatório evita tabelas e ataques com mesma password; iterations aumentam custo.
Derivar 256 bits permite dividir em key(128) + IV(128).

Q6.1

Escrever salt (16B) no início do ficheiro ao cifrar; ao decifrar, ler os 16B e refazer PBKDF2.

Q6.2

Truncado: falha no Final/unpad. Bytes iniciais em falta: key/IV errados → lixo/erro no unpad.

Q7.1

PKCS#7 via padding.PKCS7(128). dk[:16] = key; dk[16:] = IV.

Q8.1

ECB mantém contornos/padrões; CBC parece ruído (sem padrões).

Q8.2

Reutilizar IV permite ligações entre mensagens no mesmo offset e facilita análises/ataques.